RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable

Document number: P3911R0

Date: 2025-11-04

Authors: Darius Neațu <dariusn@adobe.com>, Andrei Alexandrescu <andrei@nvidia.com>, Lucian Radu Teodorescu <lucteo@lucteo.ro>, Radu Nichita <radunichita99@gmail.com>

Audience: Evolution

> **Tagline**: Contracts should describe program truth, not build configuration.

Introduction

The current C++ Contracts design allows contract assertions to be globally ignored by means of the "ignore" semantics.

In large codebases, this centralized approach to enabling contracts renders the feature effectively unreliable and non-modular — contract assertions that developers depend on for correctness can become silent no-ops depending on build configuration or project policy. This makes contracts less helpful for defining safe functions independently of build settings.

The Contracts MVP still has gaps in its design, some of which are simply missing features (such as lack of support for virtual functions). However, the specific issue of always-enabled contracts is more serious: it sets a precedent for using C++ Contracts incorrectly and introduces a teachability problem.

The Romanian NB advises addressing this issue before shipping the MVP in C++26.

Proposed Solutions

v1: Add pre! (minimal always-enabled contracts)

Introduce a minimal syntax for marking certain contracts as non-ignorable, e.g.:

```
pre (x > 0);   // subject to global configuration
pre! (y != 0); // always enforced
```

This represents a balanced compromise: it maintains configurability while ensuring safety-critical contracts are reliably checked. **No future extensions are blocked.**

There is already a substantial body of practice that motivates `pre!`, demonstrates its utility, and makes it familiar to users. Many production codebases ship an always-on assertion separate from `assert` (which follows `NDEBUG`). These macros, commonly named `CHECK`, `VERIFY`, or `ALWAYS_ASSERT`, provide the same clear specification as a precondition but cannot be elided by build configuration. They validate the condition and fail fast on violation in all builds.

Representative examples (widely used and well-known):

- *Boost:* `BOOST_VERIFY` is evaluated even in release (illustrates the desire to avoid compile-time elision; many projects layer a fatal variant atop it).
- *Abseil/Chromium:* `CHECK`, `CHECK_EQ`, … (fatal in all builds); paired with debug-only `DCHECK`.
- *glog:* `CHECK`, `PCHECK` (terminating checks; `PCHECK` also prints `errno`).
- *folly:* `CHECK`, `PCHECK`, `XCHECK` (always-on), distinct from `DCHECK`.
- *TensorFlow:* `CHECK`, `TF_CHECK_OK` (terminating, not tied to `NDEBUG`).
- *Apache Arrow:* `ARROW_CHECK`, `ARROW_CHECK_OK` (always-on guardrails).
- *gRPC:* `GPR_ASSERT` (terminates on failure in all configurations).

Note: The proposed syntax is just an example. Any syntax (or new semantics) that EWG agrees can achieve the same purpose.

v2: Remove `ignore` Semantics

With library hardening no longer tied to ignorable contracts, we can simplify the MVP by removing the ignore semantics. This prevents contracts from silently becoming no-ops. This is the smallest possible change to the draft and directly ensures that contracts cannot silently become no-ops.

Note: We can add back the ignore semantics in C++29 (together with any other extension).

Trade-off: Removing `ignore` means the language no longer offers a compile-time elidable contract. Projects that want elidable checks should continue using assert-style macros.

v3: Add (full) Contracts Properties Framework

Adopt the complete properties system proposed in P3400[1], allowing developers and implementations to control contract evaluation behavior in a flexible and extensible way.

This would be a plausible solution for C++29, but it introduces a significant design addition and may not fit within the current C++26 timeline. Moreover, it adds unneeded complexity to the standard.

v4: Move C++ Contracts to Other Shipping Vehicle

If none of the above directions reach consensus in time, another option is removing the Contracts MVP from the C++26 Working Draft and continuing its design work toward a more complete and robust solution.

This approach ensures that a newer iteration of the Contracts feature can address the desire to have mandatory contract checks, without the ability to disable them. This is important for some of the libraries that want to guarantee increased safety.

Possible future shipping vehicles:

- C++29
- A standalone White Paper
- A standalone Technical Specification (TS)

Direction Recommended by the Romanian NB

The Romanian NB strongly recommends to use the first proposal (`pre!`). We are happy with any solution EWG agrees to, and we will provide an updated wording (if changed).

Wording

Extend the grammar in [dcl.contract.func (https://eel.is/c++draft/dcl.contract.func)], then add to paragraph 1:

> A function contract assertion is unconditional if it is either
>
> - a specifier introduced by a precondition-specifier of the form
>   pre ! attribute-specifier-seq_opt (conditional-expression)
>   or
> - a postcondition specifier introduced by a postcondition-specifier of the form
>   post ! attribute-specifier-seq_opt (result-name-introducer_opt conditional-expression) .

Modify [stmt.contract.assert (https://eel.is/c++draft/stmt.contract.assert)]/1:

> An assertion-statement introduces a contract assertion ([basic.contract]). The optional attribute-specifier-seq appertains to the introduced contract assertion. An assertion-statement of the form
> contract_assert ! attribute-specifier-seq_opt (conditional-expression)
> is an unconditional contract assertion ([dcl.contract.func]).

Modify [basic.contract.eval (https://eel.is/c++draft/basic.contract.eval)]/1:

> The quick-enforce evaluation semantics is used for every evaluation of an unconditional contract assertion ([dcl.contract.func]). For any contract assertion, ~~i~~it is implementation-defined which evaluation semantics is used ~~for any given evaluation of a contract assertion~~.

References

[1] https://isocpp.org/files/papers/P3878R0.html (https://isocpp.org/files/papers/P3878R0.html)
[2] https://wg21.link/P3400 (https://wg21.link/P3400)